

DSS-Becas — Guía de estudio con capturas

Cada bloque muestra el **fragmento exacto de código** con líneas resaltadas y numeradas en naranja **1**. Debajo va la explicación de cada número, y una **pregunta probable** del docente. Las rutas indican dónde abrir el archivo en su IDE. Esta guía complementa a `docs/GUIA_ESTUDIO.md` (visión general, reparto en 5 bloques, debilidades y autoexamen).

CONTENIDO

- A. Arranque del proyecto y configuración
- B. Base de datos (Prisma)
- C. Motor DSS (lo más importante)
- D. API: rutas, seguridad y validación
- E. Controladores (lógica de cada módulo)
- F. Pruebas automáticas
- G. Frontend: estructura, sesión y datos
- H. Pantallas del sistema en funcionamiento

A. Arranque del proyecto y configuración

Lo primero que se ve al correr `npm run dev` : Vite (frontend :5173) y Express (backend :3001) arrancan juntos con `concurrently` . Estas son las piezas que lo hacen posible.

Configuración de Vite (la de tu captura)

frontend/vite.config.ts · líneas 1-24

```
vite.config.ts
frontend > vite.config.ts

1 import react from "@vitejs/plugin-react";
2 import { defineConfig } from "vite";
3
4 export default defineConfig({
5   plugins: [react()],
6   server: {
7     // 0.0.0.0: accesible desde otras PCs de la red local (ver README).
8     host: true,
9     port: 5173,
10    allowedHosts: [".localt", ".trycloudflare.com"],
11    proxy: {
12      // /api relativo: el frontend llama al backend por el mismo host/puerto.
13      "/api": { target: "http://localhost:3001", changeOrigin: true },
14    },
15  },
16  preview: {
17    host: true,
18    port: 5173,
19    proxy: {
20      "/api": { target: "http://localhost:3001", changeOrigin: true },
21    },
22  },
23 });
24
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 `host: true` → Vite escucha en 0.0.0.0, por eso en tu terminal aparecen las líneas *Network* (192.168.x.x): otras PCs de la red pueden abrir el sistema (decisión D49).
- 2 `port: 5173` → puerto fijo del frontend (D37). Coincide con `CORS_ORIGIN` del backend.
- 3 `proxy '/api'` → el navegador llama a `/api/...` en el mismo host/puerto y Vite lo reenvía a `localhost:3001` . Por eso `VITE_API_URL=/api` es relativo y funciona desde cualquier IP sin recompilar, y no hay problemas de CORS en desarrollo.

🎓 **Pregunta probable:** ¿Por qué usan un proxy en vez de llamar directo a `http://localhost:3001`? → Para que el frontend funcione desde otras máquinas de la red sin cambiar la URL, y evitar CORS.

app.ts — configuración de Express

backend/src/app.ts · líneas 1-22

```
app.ts
backend > src > app.ts

1 import cors from "cors";
2 import express from "express";
3 import { env, origenPermitido } from "../config/env";
4 import { errorHandler } from "../middlewares/errorHandler";
5 import { notFound } from "../middlewares/notFound";
6 import { router } from "../routes";
7
8 export const app = express();
9
10 app.use(
11   cors({
12     origin: (origen, cb) => {
13       if (origenPermitido(origen)) cb(null, true);
14       else cb(new Error("Origen no permitido por CORS."));
15     },
16     allowedHeaders: ["Content-Type", "Authorization"],
17   }),
18 );
19 app.use(express.json({ limit: "2mb" }));
20 app.use("/api", router);
21 app.use(notFound);
22 app.use(errorHandler);
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 CORS con función propia (`origenPermitido`): acepta el origen configurado y, con `CORS_LAN=true` , `localhost/192.168.x/10.x`.
- 2 `express.json({limit:'2mb'})` convierte el cuerpo JSON en `req.body` .
- 3 Todas las rutas cuelgan de `/api` (decisión D27).
- 4 **El orden importa:** `notFound` y luego `errorHandler` van AL FINAL para capturar lo que falle antes.

🎓 **Pregunta probable:** ¿Por qué `errorHandler` va al final? → Express ejecuta middlewares en orden; un manejador de errores solo captura lo anterior.

db.ts — un único PrismaClient y SQLite en modo WAL

backend/src/config/db.ts · líneas 1-18

```
db.ts
backend > src > config > db.ts
1 import { PrismaClient } from "@prisma/client";
2
3 export const prisma = new PrismaClient();
4
5 /**
6  * SQLite: WAL + espera de 5 s ante bloqueo (varios usuarios o Prisma Studio).
7  * Idempotente y tolerante (p. ej. base en memoria de tests).
8  */
9 async function aplicarPragmas(): Promise<void> {
10   try {
11     await prisma.$queryRawUnsafe("PRAGMA journal_mode=WAL;");
12     await prisma.$queryRawUnsafe("PRAGMA busy_timeout=5000;");
13   } catch {
14     // Sin SQLite persistente: se continúa sin WAL.
15   }
16 }
17
18 export const dbLista = aplicarPragmas();
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Una sola instancia de `PrismaClient` compartida por todo el backend.
- 2 `journal_mode=WAL` : permite leer mientras se escribe (varios usuarios, Prisma Studio abierto).
- 3 `busy_timeout=5000` : si la base está ocupada espera hasta 5 s antes de fallar (evita el 'database is locked' que causó 500 en el pasado, D55).

🎓 **Pregunta probable:** ¿Por qué SQLite y no MySQL? → No había servidor disponible; los tipos son portables y basta cambiar `provider` + `DATABASE_URL` (D26).

B. Base de datos (Prisma)

El modelo está en `backend/prisma/schema.prisma`. Son 4 tablas del SQL/SRS originales (Estudiante, Evaluacion, Resultado, Beca) más catálogos, asignaciones, seguimiento, documentos, eventos y usuarios.

Modelos Estudiante, Evaluacion, Resultado

backend/prisma/schema.prisma · líneas 15-73

```
schema.prisma

backend > prisma > schema.prisma

15 model Estudiante {
16   id_estudiante      Int           @id @default(autoincrement())
17   nombre             String
18   apellido           String
19   ci                 String?       @unique
20   fecha_nacimiento   String?
21   genero             String?
22   telefono           String?
23   correo             String?       @unique
24   direccion          String?
25   ciudad             String?
26   carrera            String
27   codigo_universitario String?     @unique
28   facultad           String?
29   semestre           Int?
30   promedio           Float
31   materias_aprobadas Int?
32   materias_reprobadas Int?
33   anio_ingreso       Int?
34   ingreso_familiar   Float
35   integrantes_hogar  Int?
36   dependientes       Int?
37   tipo_vivienda       String?
38   procedencia        String?
39   discapacidad       String?
40   situacion_laboral  String?
41   motivo             String?
42   tipo_beca_solicitada String?
43   fecha_solicitud    String?
44   evaluaciones        Evaluacion[]
45   resultados          Resultado[]
46   becas               Beca[]
47   documentos          Documento[]
48   eventos             Evento[]
49   asignaciones        Asignacion[]
50   createdAt           DateTime     @default(now())
51   updatedAt           DateTime     @updatedAt
52 }
53
54 model Evaluacion {
55   id_evaluacion      Int           @id @default(autoincrement())
56   id_estudiante      Int
57   estudiante          Estudiante @relation(fields: [id_estudiante], references: [id_estudiante], onDelete: Cascade)
58   fecha              String
59   puntaje_academico   Float
60   puntaje_social      Float
61   puntaje_final       Float
62   createdAt          DateTime     @default(now())
63
64   @@index([id_estudiante])
65 }
66
67 model Resultado {
68   id_resultado      Int           @id @default(autoincrement())
69   id_estudiante      Int           @unique
70   estudiante          Estudiante @relation(fields: [id_estudiante], references: [id_estudiante], onDelete: Cascade)
71   resultado          String
72 }
73
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 `ci`, `correo` y `codigo_universitario` son `String? @unique`: opcionales pero no repetibles (el choque da error P2002 → 409 'Ya existe un registro con ese CI').
- 2 Relación 1-a-muchos: un estudiante tiene varias evaluaciones, documentos, eventos, asignaciones.
- 3 `onDelete: Cascade`: al borrar el estudiante se borran automáticamente sus evaluaciones (y demás dependientes).

- 4 Resultado.id_estudiante @unique : solo UNA etiqueta vigente por estudiante → por eso el código usa `upsert` .

🎓 **Pregunta probable:** ¿Qué se borra al eliminar un estudiante? → En cascada: evaluaciones, resultado, becas, documentos, eventos y asignaciones.

Modelo Asignacion (beca otorgada)

📄 backend/prisma/schema.prisma · líneas 147-162

```
schema.prisma
backend > prisma > schema.prisma

147 model Asignacion {
148   id          Int          @id @default(autoincrement())
149   id_estudiante Int
150   estudiante   Estudiante   @relation(fields: [id_estudiante], references: [id_estudiante], onDelete: Cascade)
151   id_convocatoria Int
152   convocatoria Convocatoria @relation(fields: [id_convocatoria], references: [id], onDelete: Restrict) 1
153   id_tipo_beca Int
154   tipoBeca     TipoBeca     @relation(fields: [id_tipo_beca], references: [id], onDelete: Restrict)
155   puntaje      Float
156   estado       String       @default("Aprobada") 3
157   observaciones String?
158   createdAt    DateTime     @default(now())
159   seguimientos Seguimiento[]
160
161   @@unique([id_estudiante, id_convocatoria, id_tipo_beca]) 2
162 }
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Restrict : NO se puede borrar una convocatoria o tipo de beca que ya tiene asignaciones (el controlador además responde 409).
- 2 @@unique([id_estudiante, id_convocatoria, id_tipo_beca]) : impide asignar dos veces la misma beca al mismo estudiante en la misma convocatoria.
- 3 Estado por defecto 'Aprobada'; luego puede pasar a Rechazada / En observación / En espera.

🎓 **Pregunta probable:** ¿Cómo se evita duplicar asignaciones? → Restricción única compuesta en la BD, no solo validación en código.

Modelo Usuario (login)

backend/prisma/schema.prisma · líneas 178-191

```
schema.prisma
backend > prisma > schema.prisma
178 // Documentada en el ER. Login por usuario+contraseña (D57+).
179 model Usuario {
180   id_usuario      Int      @id @default(autoincrement())
181   usuario         String   @unique
182   nombre          String
183   correo          String   @unique
184   rol             String
185   activo          Boolean   @default(true)
186   password_hash   String?
187   token_version    Int      @default(0)
188   debe_cambiar_password Boolean @default(false)
189   ultimo_acceso   DateTime?
190   createdAt       DateTime @default(now())
191 }
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Nunca se guarda la contraseña, solo el hash bcrypt.
- 2 `token_version`: contador que invalida sesiones cuando cambia la contraseña/usuario.
- 3 `debe_cambiar_password`: obliga a cambiarla en el siguiente ingreso.

🎓 **Pregunta probable:** ¿Cómo cierran sesiones sin lista negra? → Subiendo `token_version`; el token viejo ya no coincide.

C. Motor DSS (lo más importante)

Vive en `backend/src/dss/` como funciones puras: no tocan la base de datos ni Express. Los controladores solo las llaman. Así se prueban fácil y se cumple la regla de arquitectura de AGENTS.md.

criterios.ts — pesos y umbrales

 `backend/src/dss/criterios.ts` · líneas 1-36

```
backend/src/dss/criterios.ts
1 // Criterios y pesos del DSS. Valores de prototypes/evaluacion-dss.png
2 // (PLAN $h). Suman 100; validarPesos() lo garantiza.
3
4 export interface CriterioDSS {
5   id: "rendimiento" | "asistencia" | "situacion" | "carga" | "vulnerable";
6   nombre: string;
7   peso: number;
8 }
9
10 export const CRITERIOS: CriterioDSS[] = [
11   { id: "rendimiento", nombre: "Rendimiento académico", peso: 30 },
12   { id: "asistencia", nombre: "Asistencia", peso: 15 },
13   { id: "situacion", nombre: "Situación socioeconómica", peso: 25 },
14   { id: "carga", nombre: "Carga familiar", peso: 15 },
15   { id: "vulnerable", nombre: "Condición vulnerable", peso: 15 },
16 ];
17
18 export const UMBRAL_RECOMENDADO = 80;
19 export const UMBRAL_REVISION = 60;
20
21 export type Recomendacion = "Recomendado" | "En revisión" | "En riesgo";
22
23 export function validarPesos(criterios: Array<{ peso: number }>): void {
24   const total = criterios.reduce((s, c) => s + c.peso, 0);
25   if (Math.abs(total - 100) > 0.001) {
26     throw new Error(`Los pesos deben sumar 100 (actual: ${total})`);
27   }
28 }
29
30 export function recomendar(puntajeFinal: number): Recomendacion {
31   if (puntajeFinal >= UMBRAL_RECOMENDADO) return "Recomendado";
32   if (puntajeFinal >= UMBRAL_REVISION) return "En revisión";
33   return "En riesgo";
34 }
35
36 validarPesos(CRITERIOS);
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Los 5 criterios y sus pesos: $30 + 15 + 25 + 15 + 15 = 100$.
- 2 Umbrales: ≥ 80 Recomendado, ≥ 60 En revisión, si no En riesgo.
- 3 `validarPesos` lanza error si los pesos no suman 100 (se ejecuta al cargar el módulo).
- 4 `recomendar(puntaje)` traduce un número a la etiqueta.

 **Pregunta probable:** ¿De dónde salen 30/15/25/15/15 y 80/60? → De `prototypes/evaluacion-dss.png` (decisión D30).

motor.ts — calcular()

backend/src/dss/motor.ts · líneas 18-46

```
motor.ts
backend > src > dss > motor.ts

18 const redondear2 = (n: number): number => Math.round(n * 100) / 100;
19
20 /** Suma ponderada con los pesos de CRITERIOS. No guarda nada. */
21 export function calcular(criterios: PuntajesCriterio): ResultadoCalculo {
22   validarPesos(CRITERIOS);
23   const peso: Record<keyof PuntajesCriterio, number> = {
24     rendimiento: 30,
25     asistencia: 15,
26     situacion: 25,
27     carga: 15,
28     vulnerable: 15,
29   };
30   const academico = redondear2((criterios.rendimiento + criterios.asistencia) / 2);
31   const social = redondear2(
32     (criterios.situacion + criterios.carga + criterios.vulnerable) / 3,
33   );
34   const final = redondear2(
35     (criterios.rendimiento * peso.rendimiento +
36       criterios.asistencia * peso.asistencia +
37       criterios.situacion * peso.situacion +
38       criterios.carga * peso.carga +
39       criterios.vulnerable * peso.vulnerable) /
40     100,
41   );
42   return {
43     puntaje_academico: academico,
44     puntaje_social: social,
45     puntaje_final: final,
46     recomendacion: recomendar(final),
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Académico = (rendimiento + asistencia) / 2.
- 2 Social = (situación + carga + vulnerable) / 3.
- 3 Final = suma ponderada $(\text{rend} \cdot 30 + \text{asist} \cdot 15 + \text{sit} \cdot 25 + \text{carga} \cdot 15 + \text{vuln} \cdot 15) / 100$.
Ejemplo 90/80/70/60/50 → **73.0** (captura de la pantalla de Evaluación).
- 4 Devuelve además la recomendación. Todo redondeado a 2 decimales.

🎓 **Pregunta probable:** Calculen 100/100/100/0/0 → académico 100, social 33.33, final 70.0 → En revisión. Ojo: los pesos aparecen fijos en el objeto `peso` (líneas 23-28), duplicando `CRITERIOS`; por eso los pesos editados en Configuración no cambian el cálculo (punto débil #1 de la guía).

motor.ts — derivarCriterios() (evaluar todos)

backend/src/dss/motor.ts · líneas 50-64

```
motor.ts
backend > src > dss > motor.ts

50 /**
51  * Criterios derivados para "evaluar todos" cuando solo hay datos básicos
52  * (D32): rendimiento = promedio; asistencia neutra 90; situación = escala
53  * inversa del ingreso (0-6000 Bs); carga y vulnerable neutros.
54  */
55 export function derivarCriterios(promedio: number, ingresoFamiliar: number): PuntajesCriterio {
56   const situacion = Math.max(0, Math.min(100, 100 - (ingresoFamiliar / 6000) * 100));
57   return {
58     rendimiento: promedio,
59     asistencia: 90,
60     situacion: redondear2(situacion),
61     carga: 70,
62     vulnerable: 60,
63   };
64 }
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Cuando solo se conocen promedio e ingreso (botón 'Evaluar pendientes') se derivan los demás criterios (D32).
- 2 Situación = escala inversa del ingreso: a más ingreso, menos puntos (0-6000 Bs → 100-0).
- 3 Asistencia 90, carga 70 y vulnerable 60 son valores neutros fijos.

🎓 **Pregunta probable:** ¿Es un dato real? → No: es una aproximación documentada para evaluar en lote.

elegibilidad.ts — filtros duros

backend/src/dss/elegibilidad.ts · líneas 3-27

```
elegibilidad.ts
backend > src > dss > elegibilidad.ts

3 // Reglas de elegibilidad previas del DSS (configurables, ver D40).
4 // promedioMinimo: escala 0-100. topeIngresoSocial: Bs mensuales máximos
5 // del ingreso familiar para becas de tipo Social.
6 export const PROMEDIO_MINIMO = 51;
7 export const TOPE_INGRESO_SOCIAL = 4000;
8
9 export interface VeredictoElegibilidad {
10   elegible: boolean;
11   motivos_no_elegible: string[];
12 }
13
14 /** Filtros duros previos al puntaje (#3). */
15 export function evaluarElegibilidad(
16   estudiante: { promedio: number; ingreso_familiar: number },
17   tipoBeca?: string,
18 ): VeredictoElegibilidad {
19   const motivos: string[] = [];
20   if (estudiante.promedio < PROMEDIO_MINIMO) {
21     motivos.push(`Promedio ${estudiante.promedio} bajo el mínimo ${PROMEDIO_MINIMO}.`);
22   }
23   if (tipoBeca === "Social" && estudiante.ingreso_familiar > TOPE_INGRESO_SOCIAL) {
24     motivos.push(`Ingreso familiar supera el tope Social (Bs ${TOPE_INGRESO_SOCIAL}).`);
25   }
26   return { elegible: motivos.length === 0, motivos_no_elegible: motivos };
27 }
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Constantes configurables: promedio mínimo 51 y tope de ingreso Bs 4000 para beca Social (D40).
- 2 Si el promedio es menor al mínimo → no elegible, con el motivo escrito.
- 3 Solo la beca Social exige el tope de ingreso. Devuelve `{elegible, motivos_no_elegible[]}`.

🎓 **Pregunta probable:** ¿Qué pasa con alguien de 95 puntos y promedio 40? → Se muestra 'No elegible' con el motivo.

ranking.ts — mejor evaluación y desempates

backend/src/dss/ranking.ts · líneas 1-28

```
ranking.ts
backend > src > dss > ranking.ts

1  export interface FilaRanking {
2    id_estudiante: number;
3    puntaje_final: number;
4    promedio: number;
5    ingreso_familiar: number;
6  }
7
8  /** Conserva solo la mejor evaluación por estudiante (evita duplicados, #1). */
9  export function mejorPorEstudiante<T extends FilaRanking>(filas: T[]): T[] {
10   const mejores = new Map<number, T>();
11   for (const f of filas) {
12     const actual = mejores.get(f.id_estudiante);
13     if (!actual || f.puntaje_final > actual.puntaje_final) mejores.set(f.id_estudiante, f);
14   }
15   return [...mejores.values()];
16 }
17
18 /**
19  * Ordena por puntaje desc, desempata por promedio desc y luego por
20  * menor ingreso familiar (más necesidad primero).
21  */
22 export function ordenarRanking<T extends FilaRanking>(filas: T[]): T[] {
23   return [...filas].sort((a, b) => {
24     if (b.puntaje_final !== a.puntaje_final) return b.puntaje_final - a.puntaje_final;
25     if (b.promedio !== a.promedio) return b.promedio - a.promedio;
26     return a.ingreso_familiar - b.ingreso_familiar;
27   });
28 }
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 mejorPorEstudiante : si hay varias evaluaciones de la misma persona, se queda con la de mayor puntaje (evita duplicados, issue #1).
- 2 Primer criterio de orden: puntaje descendente.
- 3 Desempate 1: mayor promedio.
- 4 Desempate 2: menor ingreso familiar (más necesidad primero).

🎓 **Pregunta probable:** ¿Cómo desempata el ranking? → puntaje ↓, promedio ↓, ingreso ↑.

dss/seguimiento.ts — regla de riesgo académico

backend/src/dss/seguimiento.ts · líneas 1-14

seguimiento.ts

backend > src > dss > seguimiento.ts

```
1 import { PROMEDIO_MINIMO } from "../elegibilidad";
2
3 export type EstadoSeguimiento = "Al día" | "En riesgo" | "Suspendida";
4
5 /** Regla #6: promedio del periodo bajo el mínimo → "En riesgo". */
6 export function evaluarPeriodo(promedioPeriodo: number, minimo = PROMEDIO_MINIMO): EstadoSeguimiento {
7   return promedioPeriodo < minimo ? "En riesgo" : "Al día";
8 }
9
10 /** Dos periodos seguidos en riesgo → sugerir "Suspendida". */
11 export function sugerirSuspension(estadosPrevios: string[]): boolean {
12   const ultimos = estadosPrevios.slice(-2);
13   return ultimos.length === 2 && ultimos.every((e) => e === "En riesgo");
14 }
```

1

2

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Promedio del periodo bajo 51 → 'En riesgo'; si no 'Al día'. El estado lo calcula el servidor, no lo manda el cliente.
- 2 Se sugiere suspensión cuando los **dos últimos** periodos están 'En riesgo'.

🎓 **Pregunta probable:** ¿Quién decide el estado del seguimiento? → El backend, por regla; el usuario solo digita el promedio.

D. API: rutas, seguridad y validación

Cada petición pasa por una cadena: autenticación → permiso → validación → controlador → manejador de errores.

routes/index.ts — cadena de middlewares

backend/src/routes/index.ts · líneas 38–51

```
index.ts
backend > src > routes > index.ts

38 router.get("/health", (_req, res) => res.json({ estado: "ok", fecha: new Date().toISOString() }));
39
40 // Auth pública
41 router.post("/auth/login", validate(auth.loginSchema), ah(auth.login)); 1
42
43 // Todo lo demás exige sesión + permiso explícito.
44 router.use(requireAuth); 2
45 const P = requirePermiso; 3
46 router.get("/auth/me", ah(auth.me)); 4
47 router.post("/auth/cambiar-password", validate(auth.cambiarPasswordSchema), ah(auth.cambiarPassword));
48
49 // Estudiantes
50 router.get("/estudiantes", P("estudiantes:ver"), ah(estudiantes.listar));
51 router.get("/estudiantes/:id", P("estudiantes:ver"), ah(estudiantes.obtener));
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Solo `/health` y `/auth/login` son públicas.
- 2 Desde `router.use(requireAuth)` TODAS las rutas siguientes exigen sesión.
- 3 `P(...)` es un atajo de `requirePermiso`: cada ruta declara el permiso 'modulo:accion'.
- 4 Después de esta línea se define cada endpoint.

🎓 **Pregunta probable:** ¿Qué garantiza que ninguna ruta quede sin permiso? → El test `permisos.test.ts` recorre el router y falla si falta alguno.

routes/index.ts — ejemplo de ruta completa

backend/src/routes/index.ts · líneas 57-65

index.ts

backend > src > routes > index.ts

```
57 // Evaluaciones y ranking
58 router.get("/evaluaciones", P("evaluaciones:ver"), ah(evaluaciones.listar));
59 router.get("/evaluaciones/:id", P("evaluaciones:ver"), ah(evaluaciones.obtener));
60 router.post("/evaluaciones/calcular", P("evaluaciones:crear"), validate(calcularSchema), ah(evaluaciones.calcularPuntaje)); 3 1
61 router.post("/evaluaciones/evaluar-todos", P("evaluaciones:crear"), ah(evaluaciones.evaluarTodos)); 4
62 router.post("/evaluaciones", P("evaluaciones:crear"), validate(evaluacionSchema), ah(evaluaciones.crear));
63 router.put("/evaluaciones/:id", P("evaluaciones:editar"), validate(evaluacionSchema.partial()), ah(evaluaciones.actualizar));
64 router.delete("/evaluaciones/:id", P("evaluaciones:eliminar"), ah(evaluaciones.eliminar));
65 router.get("/ranking", P("asignaciones:ver"), ah(evaluaciones.ranking));
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Cada ruta: `método + ruta + permiso + validación + controlador`.
- 2 `validate(calcularSchema)` (Zod) corta con 400 si el cuerpo es inválido.
- 3 `ah(...)` envuelve el controlador async para que los errores lleguen a `errorHandler`.
- 4 Ruta `evaluar-todos`: solo con permiso `evaluaciones:crear`.

🎓 **Pregunta probable:** ¿Qué es `ah`? → Un wrapper de handlers async, porque Express 4 no captura promesas rechazadas.

requireAuth.ts — autenticación en cada petición

backend/src/middlewares/requireAuth.ts · líneas 35-66

```
requireAuth.ts
backend > src > middlewares > requireAuth.ts

35  /** Exige JWT válido y recarga el usuario en cada petición (D58). */
36  export function requireAuth(req: Request, _res: Response, next: NextFunction): void {
37    const auth = req.headers.authorization ?? "";
38    const token = auth.startsWith("Bearer ") ? auth.slice(7) : "";
39    if (!token) {
40      next(new HttpError(401, "Sesión requerida."));
41      return;
42    }
43    let payload: { id_usuario: number; rol: string; token_version: number };
44    try {
45      payload = verificarToken(token);
46    } catch {
47      next(new HttpError(401, "Su sesión expiró o sus datos cambiaron, ingrese nuevamente."));
48      return;
49    }
50    prisma.usuario
51      .findUnique({ where: { id_usuario: payload.id_usuario }, select: SIN_HASH })
52      .then((u) => {
53        if (!u || !u.activo || u.token_version !== payload.token_version) {
54          next(new HttpError(401, "Su sesión expiró o sus datos cambiaron, ingrese nuevamente."));
55          return;
56        }
57        req.usuario = {
58          id_usuario: u.id_usuario,
59          usuario: u.usuario,
60          nombre: u.nombre,
61          correo: u.correo,
62          rol: u.rol,
63        };
64        next();
65      })
66      .catch(next);
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Lee el header `Authorization: Bearer <token>`; sin token → 401.
- 2 Verifica firma y expiración del JWT.
- 3 **Recarga al usuario desde la BD en cada petición:** los cambios de rol o estado aplican al instante (D57).
- 4 Si el usuario no existe, está inactivo o cambió `token_version` → 401.
- 5 Deja `req.usuario` disponible para `requirePermiso` y los controladores.

🎓 **Pregunta probable:** ¿Por qué consultan la BD si el token ya trae el rol? → Para que desactivar o cambiar de rol surta efecto de inmediato.

permisos.ts — matriz por rol (parte Administrador)

backend/src/auth/permisos.ts · líneas 9-33

```
permisos.ts
backend > src > auth > permisos.ts

9  export const PERMISOS_POR_ROL: Record<string, string[]> = {
10  Administrador: [
11    "estudiantes:ver",
12    "estudiantes:crear",
13    "estudiantes:editar",
14    "estudiantes:eliminar",
15    "documentos:subir",
16    "documentos:eliminar",
17    "evaluaciones:ver",
18    "evaluaciones:crear",
19    "evaluaciones:editar",
20    "evaluaciones:eliminar",
21    "asignaciones:ver",
22    "asignaciones:generar",
23    "asignaciones:decidir",
24    "asignaciones:revocar",
25    "seguimiento:ver",
26    "seguimiento:editar",
27    "reportes:ver",
28    "catalogos:ver",
29    "configuracion:ver",
30    "configuracion:editar",
31    "usuarios:gestionar",
32    "dev:reset",
33  ],
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 **Única fuente de verdad** de permisos (D61): mapa `rol → lista de 'modulo:accion'`.
- 2 Solo el Administrador gestiona usuarios.
- 3 Solo el Administrador puede restablecer datos (`dev:reset`). Evaluador y Consulta tienen listas más cortas.

🎓 **Pregunta probable:** ¿Dónde se define qué puede hacer cada rol? → Solo aquí; el frontend recibe la lista desde el servidor.

permisos.ts — requirePermiso()

backend/src/auth/permisos.ts · líneas 69-85

```
permisos.ts
backend > src > auth > permisos.ts

69  /** Exige el permiso indicado; si no → 403. Requiere requireAuth antes. */
70  export function requirePermiso(permiso: string) {
71    const mw = (req: Request, _res: Response, next: NextFunction): void => {
72      if (process.env.DSS_DEBUG_PERMISOS === "1") {
73        // eslint-disable-next-line no-console
74        console.log(`[permiso] ${req.method} ${req.path} rol=${req.usuario?.rol} permiso=${permiso}`);
75      }
76      if (!tienePermiso(req.usuario?.rol, permiso)) {
77        next(new HttpError(403, "Su rol no permite esta acción."));
78        return;
79      }
80      next();
81    };
82    // Marcado para el test de cobertura: toda ruta registrada debe llevar permiso.
83    (mw as unknown as { permiso: string }).permiso = permiso;
84    return mw;
85  }
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Comprueba si el rol del usuario incluye el permiso.
- 2 Si no → 403 'Su rol no permite esta acción.'
- 3 Marca el middleware con `.permiso` para que el test de cobertura pueda detectar rutas sin permiso.

🎓 **Pregunta probable:** ¿Diferencia entre 401 y 403? → 401: no hay sesión válida; 403: hay sesión pero el rol no alcanza.

jwt.ts — firmar y verificar tokens

backend/src/auth/jwt.ts · líneas 10-29

```
jwt.ts
backend > src > auth > jwt.ts

10 function secreto(): string {
11   const s = process.env.JWT_SECRET ?? "";
12   if (!s || s === "cambiar-en-produccion") {
13     throw new Error("JWT_SECRET no configurado.");
14   }
15   return s;
16 }
17
18 export function firmarToken(payload: PayloadJWT): string {
19   return jwt.sign(payload, secreto(), { expiresIn: "8h" });
20 }
21
22 export function verificarToken(token: string): PayloadJWT {
23   const dec = jwt.verify(token, secreto()) as Partial<PayloadJWT>;
24   if (
25     typeof dec.id_usuario !== "number" ||
26     typeof dec.rol !== "string" ||
27     typeof dec.token_version !== "number"
28   ) {
29     throw new Error("Token inválido.");
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Si `JWT_SECRET` falta o es el valor de ejemplo, el sistema **falla a propósito** en vez de correr inseguro.
- 2 El token expira en 8 horas.
- 3 `verificarToken` valida además la forma del payload (id, rol, token_version).

🎓 **Pregunta probable:** ¿Qué es un JWT? → Token firmado con el secreto del servidor que identifica al usuario sin guardar sesión en memoria.

validate.ts — validación con Zod

backend/src/middlewares/validate.ts · líneas 5-21

```
validate.ts
backend > src > middlewares > validate.ts

5  /** Valida req.body con Zod; error 400 con detalles por campo. */
6  export function validate(schema: ZodSchema) {
7    return (req: Request, _res: Response, next: NextFunction): void => {
8      const r = schema.safeParse(req.body);
9      if (!r.success) {
10         const detalles: Record<string, string> = {};
11         for (const p of r.error.issues) {
12           detalles[p.path.join(".") || "body"] = p.message;
13         }
14         next(new HttpError(400, "Datos inválidos", detalles));
15         return;
16       }
17       req.body = r.data;
18       next();
19     };
20   }
21 }
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 `safeParse` valida el cuerpo sin lanzar excepciones.
- 2 Arma `detalles {campo: mensaje}` para que el frontend pinte cada error en su campo.
- 3 Reemplaza `req.body` por los datos ya limpios/convertidos (por ejemplo `'1.499,99'` → `1499.99`).

🎓 **Pregunta probable:** ¿Por qué validar también en el servidor si ya validan en el formulario? → Nunca se confía en el cliente.

schemas.ts — aVacio y aDecimal (guardado robusto)

backend/src/validators/schemas.ts · líneas 15-36

```
schemas.ts
backend > src > validators > schemas.ts

15  /** Vacío ("", "Seleccionar...", nulos) → undefined. Nunca NaN ni "" a Prisma. */
16  function aVacio(v: unknown): unknown {
17    if (v === undefined || v === null) return undefined;
18    if (typeof v !== "string") return v;
19    const t = v.trim();
20    return t === "" || t === "Seleccionar..." ? undefined : t;
21  }
22
23  /**
24   * Decimal flexible: acepta "1499,99" y "1.499,99" como 1499.99.
25   * "" → undefined; números pasan; lo demás queda para que zod dé 400.
26   */
27  function aDecimal(v: unknown): unknown {
28    const base = aVacio(v);
29    if (base === undefined || typeof base === "number") return base;
30    if (typeof base !== "string") return base;
31    const t = base.includes(",") ? base.replace(/\.|,/, "").replace(",", ".") : base;
32    const n = Number(t);
33    return Number.isFinite(n) ? n : base;
34  }
35
36  const optStr = (schema: z.ZodString = z.string()): z.ZodType<string | undefined> =>
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 `aVacio`: convierte `''`, `'Seleccionar...'` y `null` en `undefined`. Evita guardar `''` en campos únicos (chocaban en P2002).
- 2 `aDecimal`: acepta coma decimal: `'1.499,99'` → quita puntos de miles y cambia coma por punto → `1499.99`.
- 3 Si no es número válido devuelve el texto original para que Zod dé 400; **nunca NaN a Prisma** (decisiones D51/D52).

🎓 **Pregunta probable:** ¿Qué bug resolvieron aquí? → Error 500 al guardar por coma decimal y cadenas vacías en campos únicos.

errorHandler.ts — errores uniformes

backend/src/middlewares/errorHandler.ts · líneas 22-61

```
errorHandler.ts
backend > src > middlewares > errorHandler.ts

22 export function errorHandler(
23   err: unknown,
24   _req: Request,
25   res: Response,
26   _next: NextFunction,
27 ): void {
28   if (err instanceof HttpError) {
29     res.status(err.status).json({ error: err.message, ...(err.detalles ? { detalles: err.detalles } : {}) });
30     return;
31   }
32   if (err instanceof Prisma.PrismaClientKnownRequestError) {
33     if (err.code === "P2002") {
34       const campos = (err.meta?.target as string[] | undefined) ?? [];
35       const campo = campos[0] ?? "campo";
36       const etiqueta = ETIQUETA_UNICO[campo] ?? campo;
37       res.status(409).json({
38         error: `Ya existe un registro con ese ${etiqueta}`,
39         detalles: { [campo]: `Ya existe un registro con ese ${etiqueta}` },
40       });
41       return;
42     }
43     if (err.code === "P2025") {
44       res.status(404).json({ error: "Registro no encontrado" });
45       return;
46     }
47     if (err.code === "P2003") {
48       res.status(400).json({ error: "Referencia inválida: el registro relacionado no existe." });
49       return;
50     }
51     if (err.code === "P2034" || /database is locked/i.test(err.message)) {
52       res.status(503).json({ error: "Base de datos ocupada, reintente en unos segundos." });
53       return;
54     }
55   }
56   if (err instanceof Prisma.PrismaClientValidationError) {
57     res.status(400).json({ error: "Datos inválidos", detalles: { body: err.message.split("\n").slice(0, 3).join(" ") } });
58     return;
59   }
60   const codigo = `ERR-${Date.now().toString(36).toUpperCase()}`;
61   // eslint-disable-next-line no-console
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Errores propios → su status y `{error, detalles}`.
- 2 P2002 (valor único repetido) → **409** 'Ya existe un registro con ese CI'.
- 3 P2025 (no existe) → 404.
- 4 Base bloqueada → **503** 'Base de datos ocupada'. Lo demás cae a 500 con código `ERR-XXXX` registrado en el log.

 **Pregunta probable:** ¿Qué formato tienen los errores? → Siempre JSON `{error, detalles?}`.

E. Controladores (lógica de cada módulo)

Los controladores reciben `req/res`, usan Prisma y llaman al motor DSS. Aquí están los puntos que más se preguntan.

evaluaciones.ts — calcular y guardar

backend/src/controllers/evaluaciones.ts · líneas 9–44

```
evaluaciones.ts
backend > src > controllers > evaluaciones.ts

 9  /** POST /api/evaluaciones/calcular (motor DSS + elegibilidad, sin guardar). */
10  export async function calcularPuntaje(req: Request, res: Response): Promise<void> {
11    const r = calcular(req.body.criterios);
12    const est = await prisma.estudiante.findUnique({
13      where: { id_estudiante: req.body.id_estudiante },
14      select: { promedio: true, ingreso_familiar: true },
15    });
16    if (!est) throw new HttpError(404, "Estudiante no encontrado");
17    const veredicto = evaluarElegibilidad(est, req.body.tipo_beca);
18    res.json({
19      id_estudiante: req.body.id_estudiante,
20      ...r,
21      recomendacion: veredicto.elegible ? r.recomendacion : "No elegible",
22      elegible: veredicto.elegible,
23      motivos_no_elegible: veredicto.motivos_no_elegible,
24    });
25  }
26
27  /** POST /api/evaluaciones (guarda + actualiza resultado + evento). */
28  export async function crear(req: Request, res: Response): Promise<void> {
29    const ev = await prisma.evaluacion.create({ data: req.body });
30    const etiqueta = recomendar(ev.puntaje_final);
31    await prisma.resultado.upsert({
32      where: { id_estudiante: ev.id_estudiante },
33      create: { id_estudiante: ev.id_estudiante, resultado: etiqueta },
34      update: { resultado: etiqueta },
35    });
36    await prisma.evento.create({
37      data: { id_estudiante: ev.id_estudiante, tipo: "evaluacion", detalle: `Evaluación registrada con puntaje ${ev.puntaje_final}` },
38    });
39    res.status(201).json(ev);
40  }
41
42  /** GET /api/evaluaciones */
43  export async function listar(_req: Request, res: Response): Promise<void> {
44    const items = await prisma.evaluacion.findMany({
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 `calcularPuntaje` llama al **motor** (no calcula en el controlador).
- 2 Busca promedio e ingreso del estudiante y aplica elegibilidad.
- 3 Si no es elegible la recomendación se reemplaza por 'No elegible'. **No guarda nada.**
- 4 `crear` guarda la Evaluación...
- 5 ...hace `upsert` del Resultado (crear o actualizar la única etiqueta vigente)...
- 6 ...y registra un Evento en el historial del estudiante.

 **Pregunta probable:** Observación honesta: `crear` guarda `recomendar(puntaje)` sin considerar elegibilidad (ver sección de puntos débiles).

asignaciones.ts — generar (top-N por cupos)

backend/src/controllers/asignaciones.ts · líneas 50-96

asignaciones.ts

backend > src > controllers > asignaciones.ts

```
50 /** POST /api/asignaciones/generar {convocatoriaId, tipoBecaId} (top-N por cupos). */
51 export async function generar(req: Request, res: Response): Promise<void> {
52   const { convocatoriaId, tipoBecaId } = req.body as { convocatoriaId: number; tipoBecaId: number };
53   const tipo = await prisma.tipoBeca.findUnique({ where: { id: tipoBecaId } });
54   if (!tipo) throw new HttpError(404, "Tipo de beca no encontrado.");
55   const existentes = new Set(
56     (await prisma.asignacion.findMany({ where: { id_convocatoria: convocatoriaId, id_tipo_beca: tipoBecaId } })).map(
57       (a) => a.id_estudiante,
58     ),
59   );
60   const ocupados = existentes.size;
61   const libres = Math.max(0, tipo.cupos - ocupados);
62   const evals = await prisma.evaluacion.findMany({ include: { estudiante: true } });
63   const mejores = new Map<number, (typeof evals)[number]>();
64   for (const ev of evals) {
65     const actual = mejores.get(ev.id_estudiante);
66     if (!actual || ev.puntaje_final > actual.puntaje_final) mejores.set(ev.id_estudiante, ev);
67   }
68   const candidatos = ordenarRanking(
69     [...mejores.values()]
70     .filter((ev) => !existentes.has(ev.id_estudiante))
71     .map((ev) => ({
72       id_estudiante: ev.id_estudiante,
73       puntaje_final: ev.puntaje_final,
74       promedio: ev.estudiante.promedio,
75       ingreso_familiar: ev.estudiante.ingreso_familiar,
76     })),
77   ).slice(0, libres);
78   const creadas = [];
79   for (const c of candidatos) {
80     creadas.push(
81       await prisma.asignacion.create({
82         data: {
83           id_estudiante: c.id_estudiante,
84           id_convocatoria: convocatoriaId,
85           id_tipo_beca: tipoBecaId,
86           puntaje: c.puntaje_final,
87           estado: "Aprobada",
88         },
89       }),
90     );
91     await prisma.evento.create({
92       data: { id_estudiante: c.id_estudiante, tipo: "edicion", detalle: `Beca asignada (${tipo.nombre})` },
93     });
94   }
95   res.status(201).json({ generadas: creadas.length, asignaciones: creadas });
96 }
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Cupos libres = cupos del tipo de beca – ya asignados.
- 2 Toma la mejor evaluación de cada estudiante que aún no tiene asignación y la ordena con las reglas de desempate del motor.
- 3 `slice(0, libres)` : **nunca supera los cupos.**
- 4 Crea cada asignación como 'Aprobada' y registra el evento.

 **Pregunta probable:** ¿Cómo evitan pasarse de cupos? → Solo se toman cupos – ocupados candidatos; además hay restricción única en la BD.

asignaciones.ts — decisión con observaciones

backend/src/controllers/asignaciones.ts · líneas 98–113

asignaciones.ts

backend > src > controllers > asignaciones.ts

```
98  /** PUT /api/asignaciones/:id {estado, observaciones?} (#2). */
99  export async function actualizar(req: Request, res: Response): Promise<void> {
100    const estado = String(req.body.estado ?? "Aprobada");
101    const observaciones = req.body.observaciones as string | undefined;
102    // #2: rechazar u observar exige justificación de al menos 10 caracteres.
103    if (
104      (estado === "Rechazada" || estado === "En observación") &&
105      (!observaciones || observaciones.trim().length < 10)
106    ) {
107      throw new HttpError(400, "Se requieren observaciones (mínimo 10 caracteres).", {
108        observaciones: "Se requieren observaciones (mínimo 10 caracteres).",
109      });
110    }
111    const a = await prisma.asignacion.update({
112      where: { id: Number(req.params.id) },
113      data: { estado, ...(observaciones !== undefined ? { observaciones } : {}) },
```

1

2

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Rechazar o poner 'En observación' exige justificar.
- 2 Mínimo 10 caracteres de observación, si no → 400 con el error en el campo `observaciones` (issue #2 de QA).

🎓 **Pregunta probable:** ¿Por qué obligan la justificación? → Trazabilidad de decisiones que afectan a un estudiante.

seguimiento.ts — el estado se calcula por regla

backend/src/controllers/seguimiento.ts · líneas 45-66

seguimiento.ts

backend > src > controllers > seguimiento.ts

```
45  /** POST /api/seguimiento (el estado se calcula por regla; cada cambio va a Evento). */
46  export async function crear(req: Request, res: Response): Promise<void> {
47    const asig = await prisma.asignacion.findUnique({ where: { id: Number(req.body.id_asignacion) } });
48    if (!asig) throw new HttpError(404, "Asignación no encontrada");
49    const estado = evaluarPeriodo(Number(req.body.promedio_periodo));
50    const seg = await prisma.seguimiento.create({
51      data: {
52        id_asignacion: asig.id,
53        fecha: String(req.body.fecha ?? new Date().toISOString().slice(0, 10)),
54        periodo: String(req.body.periodo),
55        promedio_periodo: Number(req.body.promedio_periodo),
56        estado,
57        observaciones: req.body.observaciones ?? null,
58      },
59    });
60    await prisma.evento.create({
61      data: { id_estudiante: asig.id_estudiante, tipo: "edicion", detalle: `Seguimiento ${seg.periodo}: ${estado}`,
62    });
63    res.status(201).json(seg);
64  }
65
66  /** PUT /api/seguimiento/:id */
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 El estado NO viene del cliente: se calcula con `evaluarPeriodo` (regla del motor).
- 2 Se guarda junto al promedio del periodo.
- 3 Cada cambio queda en el historial (Evento).

 **Pregunta probable:** ¿Se puede falsear el estado? → No, el cliente solo envía el promedio.

auth.ts — límite de intentos de login

backend/src/controllers/auth.ts · líneas 33-48

```
auth.ts
backend > src > controllers > auth.ts
33 // 5 intentos fallidos en 10 min por usuario+IP → 429.
34 const intentos = new Map<string, number[]>();
35
36 function fallosRecientes(clave: string): number {
37   const ahora = Date.now();
38   const lista = (intentos.get(clave) ?? []).filter((t) => ahora - t < 10 * 60 * 1000);
39   intentos.set(clave, lista);
40   return lista.length;
41 }
42
43 function registrarFallo(clave: string): number {
44   const lista = [...(intentos.get(clave) ?? []), Date.now()];
45   intentos.set(clave, lista);
46   return lista.length;
47 }
48
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Mapa en memoria `usuario:IP → marcas de tiempo` de fallos.
- 2 Solo cuenta los fallos de los últimos 10 minutos; con 5 → 429 'Demasiados intentos'.
- 3 Limitación conocida: se pierde al reiniciar el servidor (D58).

🎓 **Pregunta probable:** ¿Qué ataque previene? → Fuerza bruta de contraseñas.

auth.ts — login: bcrypt y token

backend/src/controllers/auth.ts · líneas 69-94

```
auth.ts
backend > src > controllers > auth.ts

69  const validar = u?.password_hash
70  ? bcryptjs.compareSync(password, u.password_hash)
71  : false;
72  if (!u || !validar) {
73    const n = registrarFallo(llave);
74    await eventoAuth(`Acceso fallido: ${clave}`);
75    if (n >= 5) {
76      throw new HttpError(429, "Demasiados intentos. Reintente en 10 minutos.");
77    }
78    throw new HttpError(401, "Usuario o contraseña incorrectos.");
79  }
80  if (!u.activo) {
81    throw new HttpError(403, "Usuario desactivado.");
82  }
83  limpiarFallos(llave);
84  await prisma.usuario.update({
85    where: { id_usuario: u.id_usuario },
86    data: { ultimo_acceso: new Date() },
87  });
88  await eventoAuth(`Acceso correcto: ${u.usuario}`);
89  const { password_hash: _h, token_version, ...resto } = u;
90  void _h;
91  res.json({
92    token: firmarToken({ id_usuario: u.id_usuario, rol: u.rol, token_version }),
93    usuario: { ...resto, permisos: permisosDe(u.rol) },
94  });
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Compara la contraseña con el hash bcrypt (nunca en claro).
- 2 Mismo mensaje para usuario inexistente o contraseña mala: **no revela cuál falló**.
- 3 Usuario desactivado → 403.
- 4 Devuelve el token y el usuario **sin** `password_hash`, con su lista de permisos.

🎓 **Pregunta probable:** ¿Por qué el mismo mensaje en ambos casos? → Para no dar pistas a un atacante.

usuarios.ts — proteger al último administrador

backend/src/controllers/usuarios.ts · líneas 186-199

```
usuarios.ts
backend > src > controllers > usuarios.ts

186 async function protegerAdmin(id: number, yo: number, pasarA: boolean): Promise<void> {
187   if (id === yo && !pasarA) {
188     throw new HttpError(400, "No puedes desactivarte ni eliminarte a ti mismo.");
189   }
190   if (!pasarA) {
191     const u = await prisma.usuario.findUnique({ where: { id_usuario: id } });
192     if (u?.rol === "Administrador") {
193       const activos = await prisma.usuario.count({ where: { rol: "Administrador", activo: true, NOT: { id_usuario: id } } });
194       if (activos === 0) {
195         throw new HttpError(400, "No se puede eliminar ni desactivar al último Administrador activo.");
196       }
197     }
198   }
199 }
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Nadie puede desactivarse ni eliminarse a sí mismo.
- 2 Si el usuario afectado es Administrador...
- 3 ...y no queda otro admin activo → se rechaza. Evita dejar el sistema sin administradores.

🎓 **Pregunta probable:** ¿Qué pasaría sin esta regla? → Un admin podría bloquear el sistema por error.

documentos.ts — subida segura con multer

backend/src/controllers/documentos.ts · líneas 9-27

```
documentos.ts
backend > src > controllers > documentos.ts

 9  const PERMITIDOS = ["application/pdf", "image/jpeg", "image/png"];
10  const MAX_BYTES = 5 * 1024 * 1024;
11
12  const storage = multer.diskStorage({
13    destination: (req, _file, cb) => {
14      const id = String((req.params as Record<string, string>).id ?? "tmp");
15      try {
16        // La carpeta debe existir ANTES de que multer escriba (si no: ENOENT → 500).
17        mkdirSync(path.join(env.UPLOAD_DIR, id), { recursive: true });
18        cb(null, path.join(env.UPLOAD_DIR, id));
19      } catch (e: unknown) {
20        cb(e as Error, "");
21      }
22    },
23    filename: (_req, file, cb) => {
24      cb(null, `${Date.now()}-${file.originalname}`);
25    },
26  });
27
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Solo PDF, JPG y PNG.
- 2 Máximo 5 MB por archivo.
- 3 La carpeta `uploads/<id>` se crea **antes** de que multer escriba: antes su ausencia causaba error 500 (ENOENT, D51).

🎓 **Pregunta probable:** ¿Cómo se guardan los archivos? → En disco; la BD solo guarda nombre, tamaño, tipo y ruta.

F. Pruebas automáticas

Vitest + supertest. Tras instalar dependencias en esta revisión se ejecutaron: **backend 52/52 y frontend 11/11 pasan.**

permisos.test.ts — cobertura de todas las rutas

backend/tests/permisos.test.ts · líneas 92-116

```
permisos.test.ts
backend > tests > permisos.test.ts

 92  it("toda ruta registrada tiene permiso (salvo públicas)", async () => {
 93    // Import dinámico: un import estático evaluaría config/db antes de
 94    // fijar DATABASE_URL y apuntaría a dev.db (D62).
 95    const { router } = await import("../src/routes");
 96    const PUBLICAS = ["/health", "/auth/login", "/auth/me", "/auth/cambiar-password"];
 97    const sinPermiso: string[] = [];
 98    const stack = (router.stack ?? []) as Array<{
 99      route?: { path: string; methods: Record<string, boolean>; stack: Array<{ handle?: unknown }> };
100    }>;
101    for (const layer of stack) {
102      const route = layer.route;
103      if (!route) continue;
104      const metodos = Object.keys(route.methods)
105        .filter((m) => route.methods[m] && m !== "_all")
106        .map((m) => m.toUpperCase())
107        .join(",");
108      const tiene = route.stack.some(
109        (l) => typeof (l.handle as { permiso?: unknown } | undefined)?.permiso === "string",
110      );
111      if (!tiene && !PUBLICAS.includes(route.path)) {
112        sinPermiso.push(`${metodos} ${route.path}`);
113      }
114    }
115    expect(sinPermiso).toEqual([]);
116  });
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Import dinámico:** un import estático crearía PrismaClient antes de fijar `DATABASE_URL` y escribiría en la base real (incidente D62/D63).
- 2** Recorre todas las rutas registradas en el router de Express.
- 3** Falla si alguna ruta (salvo las públicas) no tiene middleware de permiso.

🎓 **Pregunta probable:** ¿Qué aprendieron del incidente? → No importar estáticamente la cadena `src` en tests; usar import dinámico.

api.test.ts — pruebas del motor

backend/tests/api.test.ts · líneas 32-55

```
api.test.ts
backend > tests > api.test.ts

32 describe("motor DSS", () => {
33   it("calcula suma ponderada 30/15/25/15/15", () => {
34     const r = calcular({ rendimiento: 90, asistencia: 80, situacion: 70, carga: 60, vulnerable: 50 });
35     expect(r.puntaje_academico).toBe(85);
36     expect(r.puntaje_social).toBeCloseTo(60, 5);
37     expect(r.puntaje_final).toBe(73);
38     expect(r.recomendacion).toBe("En revisión");
39   });
40
41   it("umbrales: 80 Recomendado, 60 revisión, menos riesgo", () => {
42     expect(calcular({ rendimiento: 90, asistencia: 90, situacion: 90, carga: 90, vulnerable: 90 }).recomendacion).toBe("Recomendado");
43     expect(calcular({ rendimiento: 10, asistencia: 10, situacion: 10, carga: 10, vulnerable: 10 }).recomendacion).toBe("En riesgo");
44   });
45
46   it("ranking desempata por promedio y luego por menor ingreso", () => {
47     const filas = ordenarRanking([
48       { id_estudiante: 1, puntaje_final: 80, promedio: 70, ingreso_familiar: 5000 },
49       { id_estudiante: 2, puntaje_final: 80, promedio: 90, ingreso_familiar: 5000 },
50       { id_estudiante: 3, puntaje_final: 80, promedio: 90, ingreso_familiar: 1000 },
51     ]);
52     expect(filas.map((f) => f.id_estudiante)).toEqual([3, 2, 1]);
53   });
54 });
55
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Verifica la suma ponderada 30/15/25/15/15 (equivale a CP-002).
- 2 Verifica los umbrales 80/60.
- 3 Verifica el orden y los desempates del ranking.

🎓 **Pregunta probable:** ¿Qué exige el SRS sobre el cálculo? → Que se valide contra casos de prueba estandarizados.

G. Frontend: estructura, sesión y datos

React + TypeScript. Las páginas no llaman `fetch` directamente: usan la capa `services/api` y TanStack Query.

main.tsx — providers anidados

frontend/src/main.tsx · líneas 14-25

```
main.tsx
frontend > src > main.tsx

14  createRoot(root).render(
15    <StrictMode>
16      <QueryClientProvider client={queryClient}>
17        <ToastProvider>
18          <AuthProvider>
19            <RouterProvider router={router} />
20          </AuthProvider>
21        </ToastProvider>
22      </QueryClientProvider>
23    </StrictMode>,
24  );
25
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Caché de datos del servidor disponible en toda la app.
- 2 Notificaciones (toasts) globales.
- 3 Sesión y permisos del usuario (`useAuth`).
- 4 Router en el nivel más interno: todas las páginas pueden usar los tres contextos anteriores.

🎓 **Pregunta probable:** ¿Por qué ese orden? → Los de afuera están disponibles para los de adentro.

router.tsx — rutas anidadas y protegidas

frontend/src/routing/router.tsx · líneas 19-63

```
router.tsx
frontend > src > routing > router.tsx

19 export const router = createBrowserRouter([
20   { path: ROUTES.login, element: <LoginPage /> },
21   {
22     element: <RequireAuth />,
23     children: [
24       { path: ROUTES.cambiarPassword, element: <CambiarPasswordPage /> },
25       {
26         element: <AppLayout />,
27         children: [
28           { index: true, element: <Navigate to={ROUTES.dashboard} replace /> },
29           { path: ROUTES.dashboard, element: <DashboardPage /> },
30           { path: ROUTES.estudiantes, element: <EstudiantesPage /> },
31           {
32             element: <RequirePermiso permiso="estudiantes:crear" />,
33             children: [{ path: ROUTES.nuevoEstudiante, element: <NuevoEstudiantePage /> }],
34           },
35           { path: ROUTES.detalleEstudiante(), element: <DetalleEstudiantePage /> },
36           {
37             element: <RequirePermiso permiso="estudiantes:editar" />,
38             children: [{ path: ROUTES.editarEstudiante(), element: <NuevoEstudiantePage /> }],
39           },
40           {
41             element: <RequirePermiso permiso="evaluaciones:crear" />,
42             children: [
43               { path: ROUTES.nuevaEvaluacion, element: <EvaluacionPage /> },
44               { path: ROUTES.evaluacionPorId(), element: <EvaluacionPage /> },
45             ],
46           },
47           { path: ROUTES.becas, element: <BecasPage /> },
48           {
49             element: <RequirePermiso permiso="configuracion:ver" />,
50             children: [{ path: ROUTES.configuracion, element: <ConfiguracionPage /> }],
51           },
52           { path: ROUTES.seguimiento, element: <SeguimientoPage /> },
53           { path: ROUTES.reportes, element: <ReportesPage /> },
54           {
55             element: <RequirePermiso permiso="usuarios:gestionar" />,
56             children: [{ path: ROUTES.administracion, element: <AdministracionPage /> }],
57           },
58           { path: "**", element: <NotFoundPage /> },
59         ],
60       },
61     ],
62   },
63 ]);
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 `/login` es la única ruta pública.
- 2 `RequireAuth` : sin sesión redirige a login.
- 3 `AppLayout` pinta menú lateral + cabecera y las páginas entran por `<Outlet/>` .
- 4 `RequirePermiso` : aunque escriban la URL a mano, sin el permiso ven 'Acceso denegado'.
- 5 Cualquier otra ruta → página 404.

Pregunta probable: ¿Es seguro ocultar rutas en el frontend? → Es solo UX; la seguridad real es el 403 del backend.

guards.tsx — RequireAuth

frontend/src/routing/guards.tsx · líneas 7-26

```
guards.tsx
frontend > src > routing > guards.tsx

7  /** Exige sesión; si debe cambiar contraseña, lo fuerza antes de todo. */
8  export function RequireAuth() {
9    const auth = useAuth();
10   const location = useLocation();
11   if (auth.cargando) {                                1
12     return (
13       <div className="page">
14         <Spinner texto="Verificando sesión..." />
15       </div>
16     );
17   }
18   if (!auth.usuario) {
19     return <Navigate to={ROUTES.login} replace state={{ desde: location.pathname }} />;          2
20   }
21   if (auth.usuario.debe_cambiar_password && location.pathname !== ROUTES.cambiarPassword) {          3
22     return <CambiarPasswordPage obligatorio />;
23   }
24   return <Outlet />;
25 }
26
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Mientras se valida el token guardado muestra un spinner (evita parpadeo de login).
- 2 Sin usuario → redirige a `/login` recordando la ruta de origen.
- 3 Si `debe_cambiar_password` fuerza la pantalla de cambio antes de cualquier otra.

🎓 **Pregunta probable:** ¿Qué pasa con la primera sesión de un usuario nuevo? → Puede ser forzado a cambiar su contraseña.

client.ts — cliente HTTP

frontend/src/services/api/client.ts · líneas 29-61

```
client.ts
frontend > src > services > api > client.ts

29 async function request<T>(path: string, init?: RequestInit): Promise<T> {
30   let res: Response;
31   const cabeceras: Record<string, string> = { "Content-Type": "application/json", ...((init?.headers ?? {}) as Record<string, string>);
32   try {
33     const token = localStorage.getItem("dss-becas-token");
34     if (token) cabeceras.Authorization = `Bearer ${token}`; ❶
35   } catch {
36     // Sin acceso a localStorage: se sigue sin token.
37   }
38   try {
39     res = await fetch(`${API_BASE_URL}${path}`, { ...init, headers: cabeceras });
40   } catch {
41     throw new Error("No se pudo conectar con el servidor. Verifica que el backend esté en ejecución."); ❷
42   }
43   if (!res.ok) {
44     let cuerpo: CuerpoError = {};
45     try {
46       cuerpo = (await res.json()) as CuerpoError;
47     } catch {
48       // Respuesta no JSON: se usa el mensaje genérico.
49     }
50     if (res.status === 401 && typeof window !== "undefined") {
51       // Sesión inválida en cualquier pantalla: AuthContext redirige a /login.
52       window.dispatchEvent(new CustomEvent<string>("dss:sesion-expirada", { detail: cuerpo.error })); ❸
53     }
54     throw new ApiError(res.status, cuerpo.error ?? `Error HTTP ${res.status}`, cuerpo.detalles); ❹
55   }
56   if (res.status === 201 || res.status === 204) {
57     const text = await res.text();
58     return (text ? JSON.parse(text) : undefined) as T;
59   }
60   return (await res.json()) as T;
61 }
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- ❶ Adjunta el token guardado en cada petición.
- ❷ Si el servidor no responde → mensaje claro, sin fallback silencioso a datos falsos (D50).
- ❸ Ante cualquier 401 lanza un evento global; `AuthContext` lo escucha y cierra la sesión.
- ❹ Convierte el error del backend en `ApiError(status, mensaje, detalles)` para mostrar el mensaje exacto por campo.

🎓 **Pregunta probable:** ¿Dónde se centraliza el manejo de sesión expirada? → Aquí y en `AuthContext`, no en cada pantalla.

estudiantes.ts — misma firma en modo real y mock

frontend/src/services/api/estudiantes.ts · líneas 70-85

```
estudiantes.ts
frontend > src > services > api > estudiantes.ts

70  listar: async (f: FiltrosEstudiantes): Promise<Pagina<EstudianteListado>> => {
71    if (USE MOCKS) {
72      await simularRetardo();
73      return listarMock(f);
74    }
75    const p = new URLSearchParams();
76    if (f.q?.trim()) p.set("q", f.q.trim());
77    if (f.carrera) p.set("carrera", f.carrera);
78    if (f.estado) p.set("estado", f.estado);
79    p.set("orden", f.orden ?? "codigo");
80    p.set("dir", f.dir ?? "asc");
81    p.set("page", String(f.pagina ?? 1));
82    p.set("pageSize", String(f.porPagina ?? 10));
83    return apiClient.get<Pagina<EstudianteListado>>(`/estudiantes?${p.toString()}`);
84  },
85
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Con `VITE_USE MOCKS=true` responde desde `localStorage` (modo demo). Por defecto está apagado.
- 2 En modo real arma la query string con los filtros.
- 3 Devuelve la misma forma `{data,total,page,pageSize}`, así las páginas no saben de dónde vienen los datos.

🎓 **Pregunta probable:** ¿Para qué sirve el modo mock? → Demostrar el frontend sin backend; nunca se activa solo.

AuthContext.tsx — cierre de sesión global

frontend/src/state/AuthContext.tsx · líneas 100-109

```
AuthContext.tsx
frontend > src > state > AuthContext.tsx

100 // 401 global (token inválido, usuario eliminado/desactivado o versión cambiada).
101 useEffect(() => {
102   const handler = (e: Event): void => {
103     const detalle = (e as CustomEvent<string>).detail;
104     salir(detalle || "Su sesión expiró o sus datos cambiaron, ingrese nuevamente.");
105   };
106   window.addEventListener("dss:sesion-expirada", handler);
107   return () => window.removeEventListener("dss:sesion-expirada", handler);
108 }, [salir]);
109
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Escucha el evento que lanza `client.ts` ante un 401.
- 2 Limpia token y usuario del `localStorage` y muestra el aviso en el login.
- 3 Se quita el listener al desmontar (evita fugas de memoria).

🎓 **Pregunta probable:** ¿Qué guarda en `localStorage`? → Token y datos del usuario (con sus permisos).

EstudiantesPage — filtros en la URL + useQuery

frontend/src/pages/EstudiantesPage.tsx · líneas 42–72

```
EstudiantesPage.tsx
frontend > src > pages > EstudiantesPage.tsx

42  const q = params.get("q") ?? "";
43  const carrera = params.get("carrera") ?? "";
44  const estado = params.get("estado") ?? "";
45  const pagina = numero(params.get("pagina"), 1);
46  const porPagina = POR_PAGINA_OPCIONES.includes(numero(params.get("porPagina"), 10))
47    ? numero(params.get("porPagina"), 10)
48    : 10;
49  const orden = (params.get("orden") ?? "codigo") as Columna;
50  const dir = params.get("dir") === "desc" ? "desc" : "asc";
51
52  const [busqueda, setBusqueda] = useState(q);
53  const busquedaDebounce = useDebounce(busqueda);
54
55  useEffect(() => {
56    setParams((prev) => {
57      const next = new URLSearchParams(prev);
58      if (busquedaDebounce.trim()) next.set("q", busquedaDebounce.trim());
59      else next.delete("q");
60      next.set("pagina", "1");
61      return next;
62    });
63    // Solo reacciona al valor con debounce (no al setter estable).
64    // eslint-disable-next-line react-hooks/exhaustive-deps
65  }, [busquedaDebounce]);
66
67  const filtros: FiltrosEstudiantes = { q, carrera, estado, orden, dir, pagina, porPagina };
68  const { data, isPending, isError, refetch } = useQuery({
69    queryKey: ["estudiantes", q, carrera, estado, pagina, porPagina, orden, dir],
70    queryFn: () => estudiantesApi.listar(filtros),
71  });
72
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Filtros, orden y página viven en la URL (`useSearchParams`): se conservan al volver desde el detalle y se pueden compartir.
- 2 `useDebounce` : espera 300 ms tras la última tecla antes de buscar.
- 3 `queryKey` incluye todos los filtros: si cambia uno, TanStack Query hace una nueva consulta y guarda cada resultado en caché.

🎓 **Pregunta probable:** ¿Qué es la queryKey? → La etiqueta de caché de la consulta.

EvaluacionPage — guardar e invalidar cachés

frontend/src/pages/EvaluacionPage.tsx · líneas 117-136

```
EvaluacionPage.tsx
frontend > src > pages > EvaluacionPage.tsx

117     mutation.mutate(payload, {
118       onSuccess: () => {
119         setRegistrada(true);
120         queryClient.invalidateQueries({ queryKey: ["dashboard"] });
121         queryClient.invalidateQueries({ queryKey: ["estudiantes"] });
122         queryClient.invalidateQueries({ queryKey: ["resultado"] });
123         queryClient.invalidateQueries({ queryKey: ["historial"] });
124         queryClient.invalidateQueries({ queryKey: ["ranking"] });
125         toast.exito("Evaluación registrada.");
126       },
127       onError: (e: unknown) => {
128         if (e instanceof ApiError) setError(e.message);
129         else setSinBackend(true);
130       },
131     });
132   };
133
134   return (
135     <div className="page">
136       <PageHeader title="Evaluación DSS - Nueva evaluación" />
```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 `useMutation` envía la evaluación al backend (POST /evaluaciones).
- 2 Tras guardar, invalida `dashboard`, `estudiantes`, `ranking`, etc.: las otras pantallas se actualizan solas.
- 3 Aviso de éxito para el usuario.
- 4 Si el servidor responde con error muestra su mensaje; si no responde, avisa que no hay conexión.

🎓 **Pregunta probable:** ¿Cómo se mantienen sincronizadas las pantallas? → Con `invalidateQueries` después de cada guardado.

Sidebar.tsx — menú según permisos

frontend/src/components/layout/Sidebar.tsx · líneas 19-27

```

Sidebar.tsx
frontend > src > components > layout > Sidebar.tsx
19  const visible: Record<string, boolean> = {
20    [ROUTES.estudiantes]: usePermiso("estudiantes:ver"),
21    [ROUTES.nuevaEvaluacion]: usePermiso("evaluaciones:crear"),
22    [ROUTES.becas]: usePermiso("asignaciones:ver"),
23    [ROUTES.seguimiento]: usePermiso("seguimiento:ver"),
24    [ROUTES.reportes]: usePermiso("reportes:ver"),
25    [ROUTES.administracion]: usePermiso("usuarios:gestionar"),
26    [ROUTES.configuracion]: usePermiso("configuracion:ver"),
27  };

```

QUÉ EXPLICAR (SIGUIENDO LOS NÚMEROS NARANJAS)

- 1 Cada ítem se muestra solo si el usuario tiene el permiso (que el backend le envió al iniciar sesión).
- 2 Administración solo para quien puede gestionar usuarios (D60).

🎓 **Pregunta probable:** ¿Ocultar el menú basta? → No: es comodidad; el backend igual devuelve 403.

H. Pantallas del sistema en funcionamiento

Capturas reales del sistema corriendo con los datos de ejemplo del seed (usuario admin). Para cada una: qué archivos la implementan y qué señalar.

Login

 `pages/LoginPage.tsx` + `state/AuthContext.tsx` + `controllers/auth.ts`

Acceso al sistema

DSS-Becas · Bienestar Universitario

Usuario

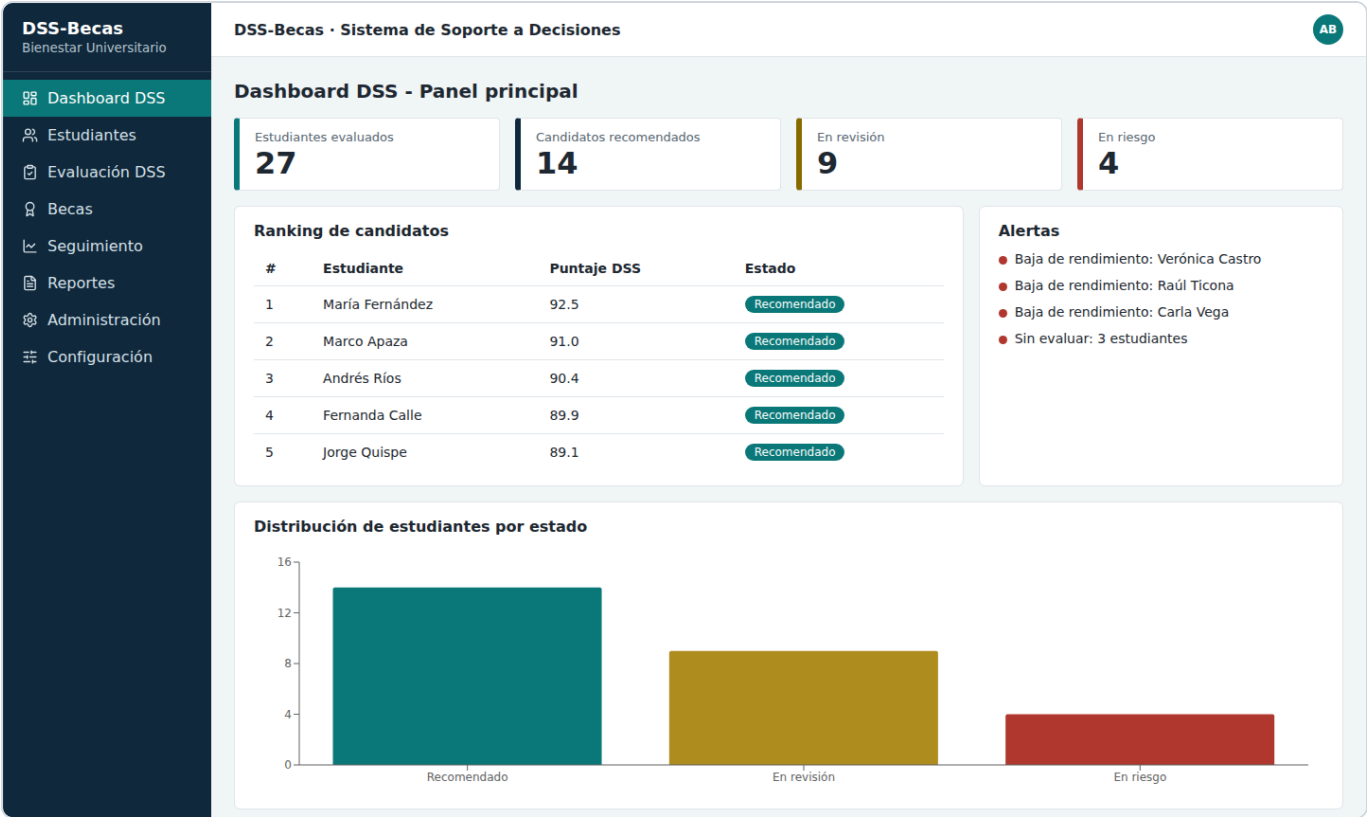
Contraseña

QUÉ EXPLICAR

- Acceso por **usuario o correo** + contraseña. Al enviar, `AuthContext.ingresar` llama `POST /api/auth/login`.
- El backend compara con bcrypt, aplica límite de 5 intentos/10 min y devuelve token JWT + permisos.
- Tras entrar vuelve a la ruta que se intentó abrir (`location.state.desde`).

Dashboard DSS

pages/DashboardPage.tsx , services/api/dashboard.ts , controllers/dashboard.ts



QUÉ EXPLICAR

- Los 4 KPI (evaluados / recomendados / en revisión / en riesgo) salen de GET /api/dashboard/resumen , calculados con consultas reales (no hardcodeados).
- Ranking top-5: **una fila por estudiante** con su mejor puntaje (issue #1).
- Alertas y gráfico de distribución (recharts/CSS). Clave de caché compartida ["dashboard"] .

Gestión de estudiantes

pages/EstudiantesPage.tsx , controllers/estudiantes.ts (listar)

DSS-Becas
Bienestar Universitario

Dashboard DSS

Estudiantes

Evaluación DSS

Becas

Seguimiento

Reportes

Administración

Configuración

DSS-Becas • Sistema de Soporte a Decisiones

Gestión de estudiantes

+ Nuevo estudiante

Buscar estudiante...

Todas las carreras

Todos los estados

Estudiantes (30)

Código	Nombre	Carrera	Estado beca	Puntaje DSS	Acción
EST-001	María Fernández	Ing. Sistemas	Activa	92.5	Ver Editar Evaluar Eliminar
EST-002	Jorge Quispe	Medicina	Activa	89.1	Ver Editar Evaluar Eliminar
EST-003	Ana Rojas	Derecho	Recomendado	81.4	Ver Editar Evaluar Eliminar
EST-004	Luis Mamani	Contaduría	En revisión	74.0	Ver Editar Evaluar Eliminar
EST-005	Carla Vega	Arquitectura	En riesgo	58.2	Ver Editar Evaluar Eliminar
EST-006	Pedro Choque	Ing. Civil	Activa	85.6	Ver Editar Evaluar Eliminar
EST-007	Sofía Aguilar	Psicología	Pendiente	-	Ver Editar Evaluar Eliminar
EST-008	Diego Condori	Ing. Sistemas	Activa	88.3	Ver Editar Evaluar Eliminar
EST-009	Lucía Huanca	Medicina	En revisión	71.2	Ver Editar Evaluar Eliminar
EST-010	Marco Apaza	Derecho	Activa	91.0	Ver Editar Evaluar Eliminar

Mostrando 1-10 de 30

10 por página

Anterior

Siguiente

QUÉ EXPLICAR

- Búsqueda con debounce, filtros de carrera/estado, orden y paginación **guardados en la URL**.
- El estado (Recomendado/En revisión/En riesgo/Pendiente/Activa) se deriva en `estadoDe()` del backend.
- Los botones (Nuevo, Editar, Evaluar, Eliminar) aparecen según los permisos del rol.

Detalle del estudiante

pages/DetalleEstudiantePage.tsx , controllers/documentos.ts , estudiantes.historial

DSS-Becas
Bienestar Universitario

Dashboard DSS

Estudiantes

Evaluación DSS

Becas

Seguimiento

Reportes

Administración

Configuración

DSS-Becas · Sistema de Soporte a Decisiones

AB

Detalle del estudiante

Volver

Editar

Evaluar con DSS

Eliminar

Ficha del estudiante

MF

María Fernández

EST-001 / Ing. Sistemas

Recomendado

Puntaje DSS

92.5

Asistencia

-

Promedio

88.0

Beca actual

Excelencia (Bs 500,00)

Evaluación DSS

Puntaje académico

Puntaje social

Puntaje final

Fecha: 01/08/2025 · Recomendación:

Recomendado

Historial de seguimiento

Sin historial registrado.

Información personal

CI

1000001

Teléfono

70000001

Ciudad

La Paz

Fecha de nacimiento

14/05/2000

Correo

maria.fernandez@universidad.bo

Género

-

Dirección

Av. Principal 123

Información académica

QUÉ EXPLICAR

- Ficha completa, evaluación vigente, documentos (subir/descargar/eliminar) e **historial de eventos** (tabla Evento).
- Cada acción del sistema (crear, editar, evaluar, asignar) deja un evento con fecha.

Alta de estudiante

pages/NuevoEstudiantePage.tsx (alta y edición), validators/schemas.ts

DSS-Becas
Bienestar Universitario

Dashboard DSS

Estudiantes

Evaluación DSS

Becas

Seguimiento

Reportes

Administración

Configuración

DSS-Becas • Sistema de Soporte a Decisiones

AB

Nuevo estudiante

Registro

Datos personales

Nombres *

Apellidos *

Carnet de identidad *

Fecha de nacimiento *

Género

Teléfono

Correo

Dirección

Ciudad / Departamento

Carrera *

Datos académicos

Código universitario

Facultad

Semestre

Promedio académico *

Materias aprobadas

Materias reprobadas

Año de ingreso

Datos socioeconómicos

Ingreso familiar mensual *

Integrantes del hogar

Dependientes

Tipo de vivienda

Procedencia

Discapacidad

Seleccionar...

Seleccionar...

Seleccionar...

QUÉ EXPLICAR

- Un solo componente sirve para **alta y edición** (según haya `:idEstudiante`).
- Valida en el cliente (mismas reglas que Zod en el servidor) y muestra los errores del servidor en cada campo (`MAPA_SERVIDOR`).
- Acepta coma decimal (`parseDecimal` en el frontend, `aDecimal` en el backend).

Evaluación DSS (73.0 En revisión)

pages/EvaluacionPage.tsx , dss/motor.ts

DSS-Becas

Bienestar Universitario

Dashboard DSS

Estudiantes

Evaluación DSS

Becas

Seguimiento

Reportes

Administración

Configuración

DSS-Becas • Sistema de Soporte a Decisiones

AB

Evaluación DSS - Nueva evaluación

Criterios ponderados

Estudiante

María Fernández

Fecha

09/29/2026

Rendimiento académico

90

30%

Asistencia

80

15%

Situación socioeconómica

70

25%

Carga familiar

60

15%

Condición vulnerable

50

15%

Calcular puntaje DSS

Resultado de la evaluación

Puntaje DSS

73.0

Recomendación: En revisión

Comparación con umbral

Recomendado (>= 80)

En revisión (60-79)

En riesgo (< 60)

QUÉ EXPLICAR

- Con 90/80/70/60/50 el resultado es **73.0 → En revisión** (mismo ejemplo de la guía).
- El botón llama primero POST /evaluaciones/calcular (motor + elegibilidad) y luego guarda con POST /evaluaciones .
- La leyenda de umbrales sale del componente UmbralLegend .

Gestión de becas

pages/BecasPage.tsx , controllers/asignaciones.ts , evaluaciones.ranking

DSS-Becas
Bienestar Universitario

Dashboard DSS

Estudiantes

Evaluación DSS

Becas

Seguimiento

Reportes

Administración

Configuración

DSS-Becas · Sistema de Soporte a Decisiones

AB

Gestión de becas

Convocatoria Becas 2025

Excelencia

Generar asignación

Evaluar pendientes

Exportar CSV

Exportar PDF

Cupos ocupados

1

Cupos disponibles

19

Presupuesto usado (Bs)

600

Presupuesto disponible (Bs)

49400

Ranking de candidatos (27)

#	Estudiante	Carrera	Puntaje	Recomendación
1	María Fernández	Ing. Sistemas	92.5	Recomendado
2	Marco Apaza	Derecho	91.0	Recomendado
3	Andrés Ríos	Economía	90.4	Recomendado
4	Fernanda Calle	Medicina	89.9	Recomendado
5	Jorge Quispe	Medicina	89.1	Recomendado
6	Diego Condori	Ing. Sistemas	88.3	Recomendado
7	Paola Vargas	Enfermería	87.2	Recomendado
8	Natalia Ortiz	Contaduría	86.1	Recomendado
9	Pedro Choque	Ing. Civil	85.6	Recomendado
10	Carmen Poma	Ing. Civil	84.5	Recomendado
11	Martín Paredes	Ing. Civil	83.7	Recomendado
12	Daniela Miranda	Ing. Sistemas	82.8	Recomendado
13	Ana Rojas	Derecho	81.4	Recomendado
14	Beatriz Mamani	Psicología	80.5	Recomendado

QUÉ EXPLICAR

- Se elige convocatoria + tipo de beca; los cuadros muestran **cupos y presupuesto** (GET /asignaciones/resumen).
- 'Generar asignación' toma los top-N según cupos libres; 'Evaluar pendientes' usa criterios derivados.
- Exportar CSV/PDF con utils/export.ts .

Seguimiento académico

pages/SeguimientoPage.tsx , dss/seguimiento.ts , controllers/seguimiento.ts

DSS-Becas
Bienestar Universitario

Dashboard DSS

Estudiantes

Evaluación DSS

Becas

Seguimiento

Reportes

Administración

Configuración

DSS-Becas · Sistema de Soporte a Decisiones

Seguimiento académico

Todos los estadosTodas las carreras

Registros (2)

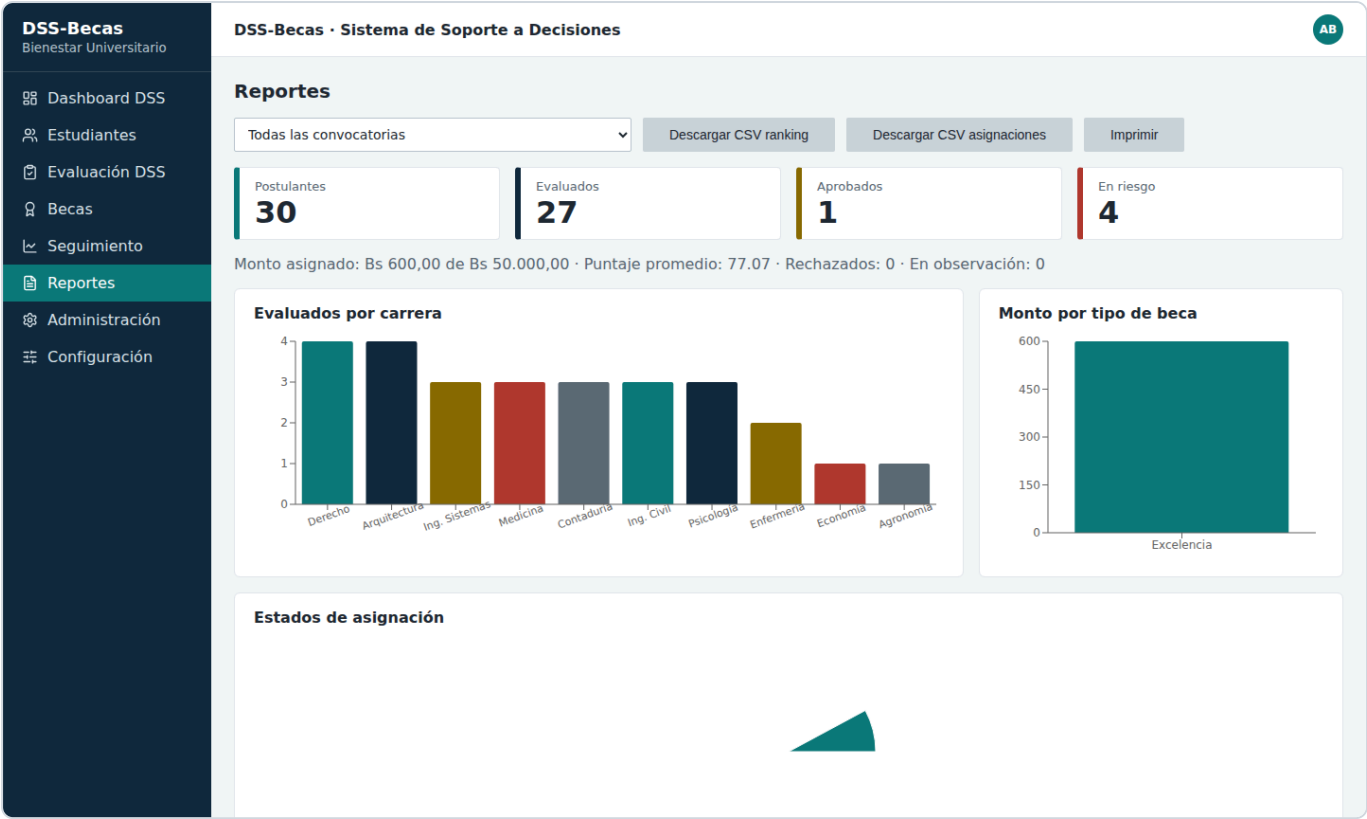
Becario	Carrera	Periodo	Promedio	Estado	Acción
María Fernández	Ing. Sistemas	2025-I	88.0	Al día	Ver historial
María Fernández	Ing. Sistemas	2024-II	86.0	Al día	Ver historial

QUÉ EXPLICAR

- Historial por periodo de cada becario; el estado se calcula por regla (promedio < 51 → En riesgo).
- Dos periodos seguidos en riesgo → sugerencia de suspensión.

Reportes

pages/ReportesPage.tsx , controllers/reportes.ts



QUÉ EXPLICAR

- Resumen por convocatoria (postulantes, aprobados, monto vs presupuesto) con gráficos recharts.
- CSV generado en el servidor con BOM UTF-8 y ; (Excel en español). Impresión con @media print .

Configuración (catálogos)

pages/ConfiguracionPage.tsx (CrudTab<T> genérico), controllers/catalogos.ts

DSS-Becas

Bienestar Universitario

Dashboard DSS

Estudiantes

Evaluación DSS

Becas

Seguimiento

Reportes

Administración

Configuración

DSS-Becas • Sistema de Soporte a Decisiones

Configuración

Carreras

Tipos de beca

Convocatorias

Criterios DSS

+ Nuevo

Buscar...

Carreras y facultades

Nombre	Facultad	Estado	Acción
Ing. Sistemas	-	Activa	Editar Eliminar
Medicina	-	Activa	Editar Eliminar
Derecho	-	Activa	Editar Eliminar
Contaduría	-	Activa	Editar Eliminar
Arquitectura	-	Activa	Editar Eliminar
Ing. Civil	-	Activa	Editar Eliminar
Psicología	-	Activa	Editar Eliminar
Enfermería	-	Activa	Editar Eliminar
Economía	-	Activa	Editar Eliminar
Agronomía	-	Activa	Editar Eliminar

QUÉ EXPLICAR

- Pestañas de carreras, tipos de beca, convocatorias y criterios con un único componente CRUD genérico.
- Guardado de pesos en lote con suma 100 validada por Zod (issue #4).
- **Ojo (punto débil):** estos pesos se guardan pero el motor DSS usa los fijos.

Administración de usuarios

pages/AdministracionPage.tsx , controllers/usuarios.ts

DSS-Becas

Bienestar Universitario

Dashboard DSS

Estudiantes

Evaluación DSS

Becas

Seguimiento

Reportes

Administración

Configuración

DSS-Becas • Sistema de Soporte a Decisiones

AB

Administración

+ Nuevo usuario

Buscar por usuario, nombre, correo o rol...

Usuarios (3)

Usuario	Nombre	Correo	Rol	Último acceso	Estado	Acción
@admin	Admin Bienestar	admin@universidad.bo	Administrador	29/09/2026 20:51	Activo	Editar Desactivar Eliminar
@evaluador	Evaluador DSS	evaluador@universidad.bo	Evaluador	-	Activo	Editar Desactivar Eliminar
@consulta	Consulta Rectorado	consulta@universidad.bo	Consulta	-	Activo	Editar Desactivar Eliminar

QUÉ EXPLICAR

- CRUD de usuarios con roles Administrador/Evaluador/Consulta y activar/desactivar.
- Protecciones: no borrarse a sí mismo ni al último admin activo.

Rol Consulta: acceso denegado

`routing/guards.tsx` (RequirePermiso), `auth/permisos.ts`



QUÉ EXPLICAR

- Entrando como **consulta** y abriendo `/administracion` por URL se ve 'Acceso denegado'.
- Y aunque se saltara la UI, el backend respondería 403 (defensa en dos capas).